Теория: векторный вход и бинарная классификация

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Содержание

Минимальное повторение перед началом **1**
Что нужно помнить о скалярном `Value` . . . 1
Мини-глоссарий . . . 2
Готовность к векторному входу . . . 2

1. Задача классификации и обозначения **3**
Свободные и немые индексы . . . 3
Зачем в работе нужны OR, AND и XOR . . . 3
Направление ребра графа и локальный вклад . . . 5

2. Векторный вход как скалярный вычислительный граф **8**
2.1. Аффинный узел и скалярное раскрытие . . . 8
2.2. Что является вершиной графа . . . 9
2.3. Как скалярный autograd становится векторной моделью . . . 10

3. Один sigmoid-нейрон: прогноз, решение и функция потерь **10**
Потеря одного объекта . . . 11
Бинарная кросс-энтропия (binary cross-entropy, BCE) . . . 11
Как читать укрупнённые схемы нейронов . . . 12

4. Аналитическое дифференцирование одного sigmoid-нейрона **14**
4.1. Sigmoid как композиция скалярных функций . . . 14
4.2. Производная BCE по вероятности . . . 15
4.3. Локальная ошибка логита и средняя потеря . . . 16
4.4. Полные производные параметров . . . 16

5. Reverse-mode получает те же градиенты **17**
5.1. Средняя потеря и seed-gradient . . . 17
5.2. BCE, sigmoid и аффинный узел одного объекта . . . 17
5.3. Общие параметры всего датасета . . . 19

6. Один скрытый нейрон: композиция и цепное правило на AND **21**

7. Зачем в маршруте нужен XOR: ограничение одного скрытого нейрона **24**

8. Два скрытых нейрона: XOR как минимальный тест выразительности **25**
8.1. Обратный проход скрытого слоя . . . 26
8.2. Итоговый цикл обучения . . . 28

9. От скалярных формул к векторам и матрицам **29**
9.1. Матрица датасета и соглашение о формах . . . 29
9.2. Один sigmoid-нейрон как матричная формула . . . 29
9.3. Скрытый слой из m tanh-нейронов . . . 29
9.4. Соответствие трёх записей . . . 30
Необязательный мостик: нотация Эйнштейна . . . 31
9.5. От XOR к произвольной MLP . . . 32

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Этот текст продолжает лабораторную работу «Скалярное автоматическое дифференцирование и обучение одного нейрона». Предыдущая работа остаётся рекомендуемым пререквизитом: в ней подробно выводятся правило цепочки, вычислительный DAG и reverse-mode. Однако перед началом нового модуля достаточно восстановить минимальную модель скалярного autograd из следующего раздела. Поэтому не требуется помнить весь предыдущий материал буквально.

Новая задача проста по формулировке, но важна по смыслу: один объект теперь содержит не одно число, а вектор признаков. Мы хотим по этому вектору предсказывать один из двух классов. При этом `Value` не становится векторным: каждая вершина графа по-прежнему хранит одно вещественное число. Векторная модель строится циклом из знакомых скалярных умножений, сложений и нелинейностей.

Маршрут лекции ведёт к трём наблюдаемым результатам лабораторной:

Модель	Учебный датасет	Наблюдаемый результат
Один sigmoid-нейрон	OR	точная классификация линейно разделимых объектов
Один скрытый tanh-нейрон и sigmoid-выход	AND	прослеженный градиент через две аффинные операции и нелинейность
Два скрытых tanh-нейрона и sigmoid-выход	XOR	точная классификация нелинейно разделимой задачи

Минимальное повторение перед началом

Этот раздел рассчитан примерно на 20–30 минут. Его цель — восстановить ровно тот механизм, который будет использоваться в лабораторной: новый класс автоматического дифференцирования здесь не создаётся. Для полного повторения при необходимости обратитесь к теоретическому и лабораторному материалам предшествующего модуля.

Что нужно помнить о скалярном `Value`

Один объект `Value` представляет одну скалярную вершину вычислительного графа. Поле `data` хранит число, полученное в прямом проходе. Поле `grad` хранит накопленный полный градиент итоговой скалярной функции по этой вершине. Ссылки на непосредственных родителей позволяют восстановить, из каких более ранних величин получен результат.

Когда Python вычисляет выражение вроде `a * a + a`, перегруженные операции создают новые вершины, сохраняют родителей и записывают локальное правило обратного прохода. В прямом направлении вычисляются числа. В обратном направлении `backward()` сначала строит топологический порядок достижимых вершин, задаёт итоговой вершине seed-gradient 1 и затем вызывает сохранённые локальные правила в обратном порядке.

Рассмотрим ту же проверку, которая первой появится в guide:

$$J = a^2 + a, \qquad a = 2.$$

Прямой проход создаёт промежуточную величину $q = a^2 = 4$ и итог $J = q + a = 6$. В обратном проходе сначала получаем

$$g_J = \frac{\partial J}{\partial J} = 1.$$

Сложение $J = q + a$ передаёт единичный вклад в q и один вклад 1 непосредственно в a. Затем узел $q = a \cdot a$ имеет две ссылки на один и тот же объект a. Его локальное правило передаёт два вклада: по 2 через первый и второй множители. Поэтому накопленный градиент равен

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

$$g_a = 1 + 2 + 2 = 5 = \frac{\partial J}{\partial a} = 2a + 1.$$

Здесь запись с несколькими знаками равенства означает одно и то же число при $a = 2$, а не последовательность присваиваний. В коде складывание независимых вкладов выполняется оператором `+=`. Именно поэтому одно повторно используемое значение может быть родителем нескольких рёбер графа без потери частей производной. Ячейка `scalar_graph_refresh` лабораторной запускает этот точный пример и проверяет $J = 6$ и $g_a = 5$.

Мини-глоссарий

Термин	Практический смысл в этой работе
Скалярная вершина `Value`	один вещественный результат операции или лист графа
Родитель	непосредственный вход операции; стрелка прямого графа идёт от родителя к результату
Прямой проход	вычисление и сохранение значений `data`
Обратный проход	распространение производных итоговой функции по сохранённому DAG в обратном топологическом порядке
Seed-gradient	начальное значение $\frac{\partial J}{\partial J} = 1$ у итоговой скалярной функции
Накопление `+=`	сложение вкладов в `grad`, если к вершине ведут несколько путей
Параметр	лист `Value`, который возвращает `parameters()` и изменяется градиентным спуском
Константный лист	число-признак или метка; формально участвует в графе, но оптимизатор его не обновляет

Готовность к векторному входу

Перед разделом 1 студент должен уметь объяснить следующие утверждения.

- `data` появляется в прямом проходе, а `grad` после `backward()` означает производную итоговой скалярной функции по данной вершине.
- Итоговый скаляр получает seed-gradient 1, потому что его производная по самому себе равна 1.
- Один объект `Value` может получить несколько вкладов; поэтому градиент складывается через `+=`, а не перезаписывается.
- Веса являются параметрами, а признаки и метки — данными: обратный проход может формально вычислить производную и по данным, но градиентный спуск обновляет только параметры.

Если хотя бы одно утверждение пока неясно, сначала выполните ячейку `scalar_graph_refresh` из guide и вернитесь к соответствующему разделу скалярной лабораторной. После этого можно переходить к векторному входу: новый материал использует те же скалярные правила, но повторяет их для координат одного объекта и для объектов датасета.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

1. Задача классификации и обозначения

Свободные и немые индексы

В общих формулах этой работы латинские индексы обозначают фиксированные компоненты: i — фиксированный объект датасета, j — фиксированную координату входа или соответствующий вес, h — фиксированный скрытый нейрон, k — фиксированную выходную или скрытую координату. Такой **свободный индекс** определяет компоненту результата и должен согласованно сохраняться в обеих частях равенства.

Греческие α, β и μ используются как **немые** (суммируемые, фиктивные) индексы. Они всегда связаны явным знаком $\sum$ или $\prod$: α — свёртка по входным координатам, β — свёртка по скрытым состояниям, μ — агрегация по объектам датасета. Немой индекс можно согласованно переименовать: $\sum_{\mu=1}^{n} \ell_\mu$ и $\sum_{\rho=1}^{n} \ell_\rho$ означают одно и то же число. Основная часть работы сохраняет явные суммы и произведения.

Греческая буква сама по себе не делает символ индексом. Здесь δ_i, σ и ℓ_i — имена величин: локальной ошибки, sigmoid и потери. В финальном обобщении ℓ остаётся традиционным свободным индексом слоя; он не является немым индексом и не связан со знаком суммы.

Зачем в работе нужны OR, AND и XOR

В лабораторной используются три булевы функции двух бинарных признаков $x_1, x_2 \in \{0, 1\}$. Их полный датасет состоит всего из четырёх объектов:

x_1	x_2	OR	AND	XOR
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

OR, AND и XOR не являются прикладными целевыми функциями нейросети. Это учебные контролируемые датасеты: у них известны все входы, все метки, геометрия классов и ожидаемый результат обучения. Поэтому на них можно отдельно проверить три разных свойства модели.

Учебный датасет	Свойство классов	Роль в маршруте
OR	линейно разделим	проверить, что один sigmoid-нейрон обучает одну аффинную границу
AND	линейно разделим	проследить правило цепочки через скрытый tanh-нейрон и выходной sigmoid
XOR	нелинейно разделим	увидеть ограничение одного скрытого признака и пользу нескольких скрытых характеристик

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

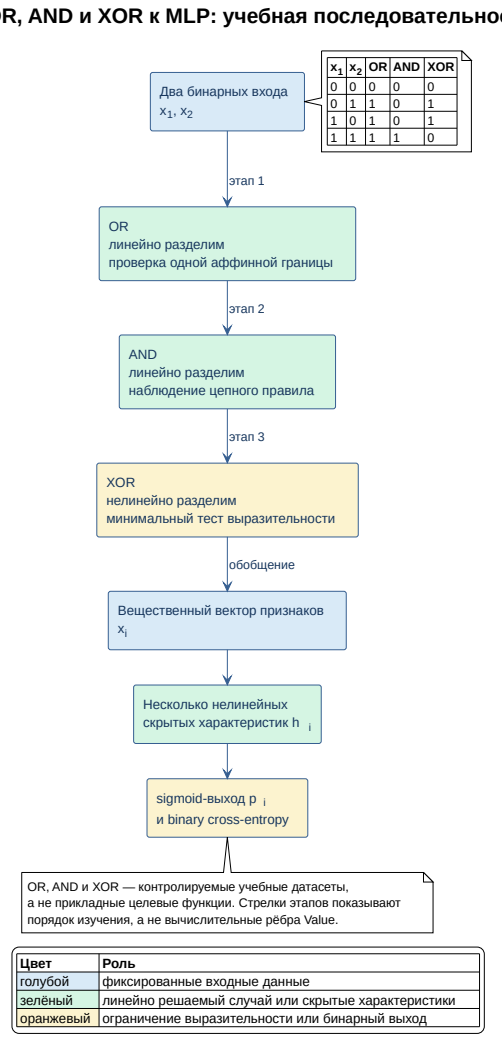


Рис. 1. От учебных булевых задач к MLP

В реальной бинарной классификации признаки обычно вещественные и их значительно больше двух: это измерения, признаки после предобработки или результаты предыдущего слоя. Булевы функции не заменяют реальные датасеты и не утверждают, что прикладная задача буквально является XOR. Они делают наблюдаемыми те же вопросы, которые возникают в многослойном перцептроне: достаточно ли одной аффинной границы, как градиент проходит через композицию и зачем слою несколько нелинейных признаков. Во всех случаях сохраняется один цикл: вектор признаков $\rightarrow$ модель $\rightarrow$ вероятность $\rightarrow$ BCE $\rightarrow$ `backward()` $\rightarrow$ обновление параметров.

Датасет содержит n объектов. У объекта с индексом i есть d числовых признаков и бинарная метка:

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id}), \qquad y_i \in \{0, 1\}.$$

В лабораторной $d = 2$. Поэтому вход одного объекта записывается как список `[x_{i1}, x_{i2}]`, а цель модели — получить вероятность класса 1:

$$p_i \in (0, 1).$$

Жёсткая метка прогноза нужна только для проверки классификации:

$$\hat{y}_i = \begin{cases} 1, & p_i \geq 0.5, \\ 0, & p_i < 0.5. \end{cases}$$

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Параметры модели будут объектами `Value`. Координаты x_{ij} и метки y_i являются фиксированными данными. При арифметической операции с `Value` обычное число Python временно оборачивается в константный лист графа. Такой лист формально может получить поле `grad`, но метод `parameters()` его не возвращает, поэтому градиентный спуск его не изменяет.

В этой работе используются следующие обозначения:

Обозначение	Смысл
o_i	логит, то есть вещественный выход аффинного узла перед sigmoid
p_i	вероятность класса 1 после sigmoid
ℓ_i	потеря одного объекта
L	средняя потеря по всему датасету
g_v	полный градиент $\frac{\partial L}{\partial v}$ в вершине v
$\Delta g_u^{(v)}$	один вклад в градиент u, пришедший через узел v

Для любого узла v скалярного графа будем писать:

$$g_v = \frac{\partial L}{\partial v}.$$

Если узел v зависит от непосредственного родителя u, обратный проход передаёт не полный градиент g_u, а один локальный вклад в него:

$$\Delta g_u^{(v)} = g_v \frac{\partial v}{\partial u}.$$

Поле `u.grad` хранит сумму всех уже пришедших вкладов:

$$g_u \leftarrow g_u + \Delta g_u^{(v)}.$$

Знак $\leftarrow$ и оператор `+=` обозначают накопление. Они не создают новую производную и не заменяют правило цепочки.

Направление ребра графа и локальный вклад

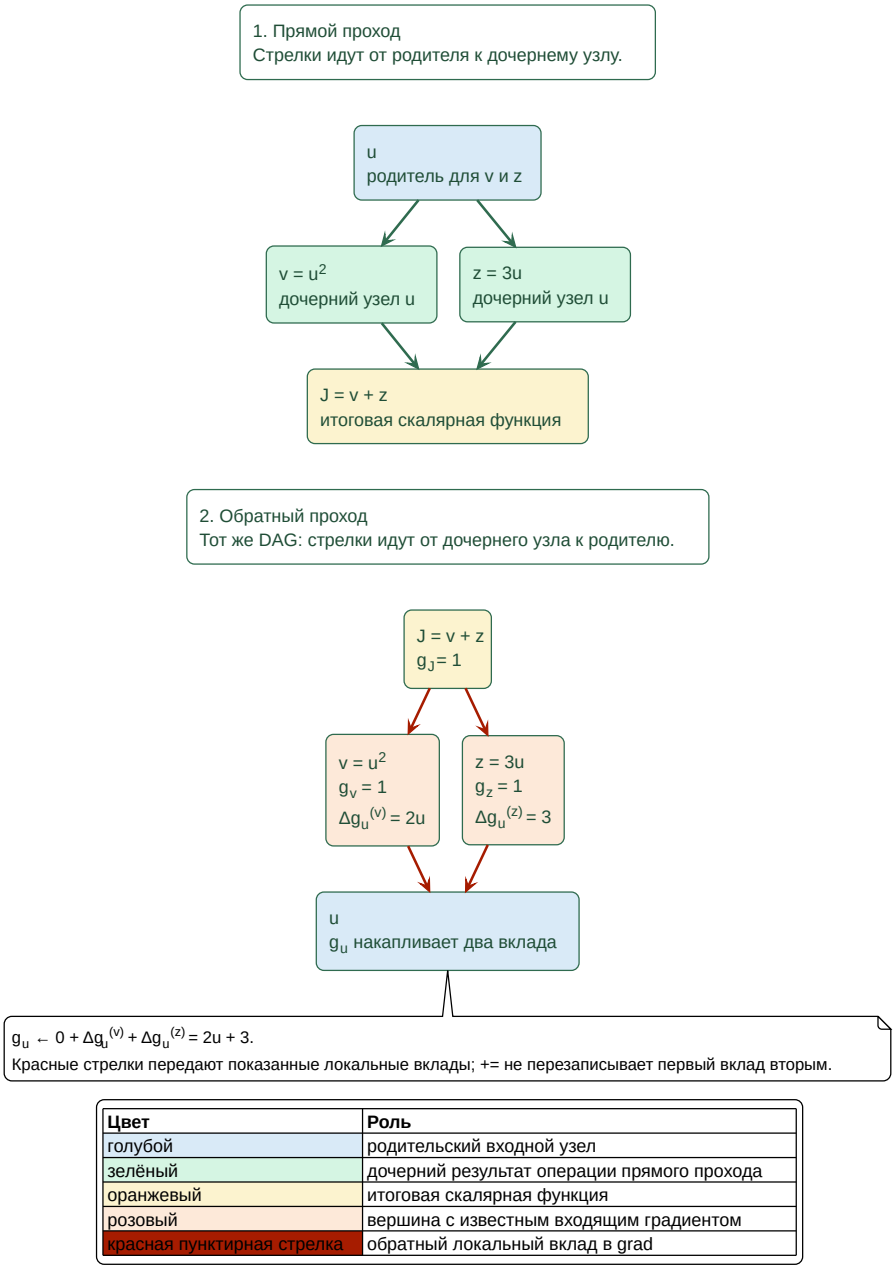
Стрелки прямого вычислительного графа направлены от входов операции к её результату. Поэтому в связи $u \to v$ узел u называют непосредственным родителем узла v, а узел v — дочерним узлом u. В обратном проходе направление движения градиентов противоположно: от дочернего узла к его родителю.

Рассмотрим небольшой ветвящийся граф:

В этой врезке J обозначает произвольную итоговую скалярную функцию и играет ту же роль, которую далее играет средняя потеря L. Поэтому внутри примера используем запись $g_a = \frac{\partial J}{\partial a}$ для каждой вершины a.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Родитель, дочерние узлы и локальные вклады



Цвет	Роль
голубой	родительский входной узел
зелёный	дочерний результат операции прямого прохода
оранжевый	итоговая скалярная функция
розовый	вершина с известным входящим градиентом
красная пунктирная стрелка	обратный локальный вклад в grad

Рис. 2. Родитель, дочерние узлы и локальные вклады

В прямом проходе u является родителем одновременно для $v = u^2$ и $z = 3u$. Узлы v и z являются двумя дочерними узлами u. Затем сложение создаёт итоговую скалярную функцию $J = v + z$.

В обратном проходе начинаем с:

$$g_J = \frac{\partial J}{\partial J} = 1.$$

Сложение передаёт одинаковые входящие градиенты своим родителям:

$$g_v = \frac{\partial J}{\partial v} = 1, \qquad g_z = \frac{\partial J}{\partial z} = 1.$$

Дочерний узел $v = u^2$ передаёт родителю u только свой локальный вклад:

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

$$\Delta g_u^{(v)} = g_v \frac{\partial v}{\partial u} = 1 \cdot 2u = 2u.$$

Второй дочерний узел $z = 3u$ передаёт независимый вклад:

$$\Delta g_u^{(z)} = g_z \frac{\partial z}{\partial u} = 1 \cdot 3 = 3.$$

Поэтому поле `u.grad` накапливает оба вклада, а не получает один из них вместо другого:

$$g_u \leftarrow 0 + \Delta g_u^{(v)} + \Delta g_u^{(z)} = 2u + 3.$$

Следующая схема показывает **один и тот же DAG с общими вершинами** в горизонтальном направлении. Зелёные сплошные стрелки проводят значения слева направо в прямом проходе. Красные пунктирные стрелки проходят по тем же рёбрам в противоположную сторону: они проводят градиенты и локальные вклады справа налево. Это не две разные структуры: обратный проход использует сохранённые вершины и рёбра прямого вычислительного графа.

Один вычислительный граф: значения — слева направо, градиенты — справа налево

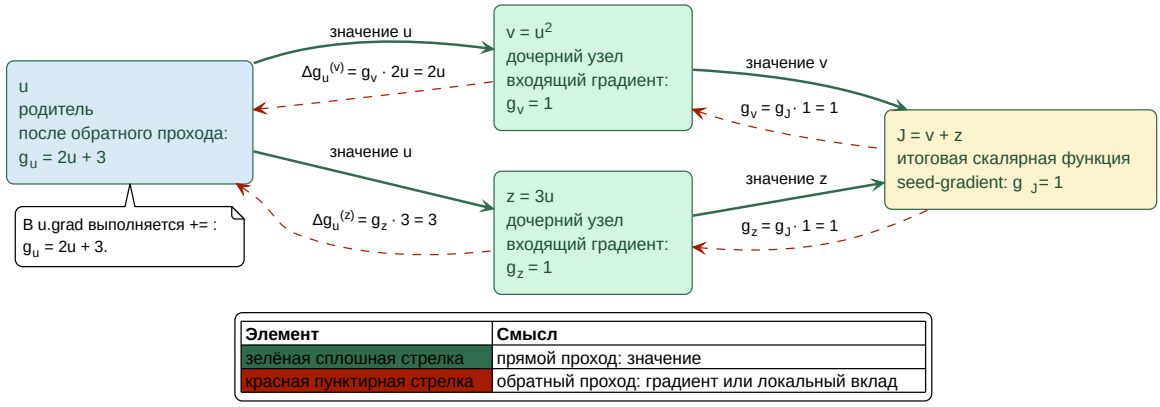


Рис. 3. Горизонтальный прямой и обратный проходы

Это совпадает с обычным дифференцированием $J = u^2 + 3u$:

$$\frac{\partial J}{\partial u} = 2u + 3.$$

Итак, обратный проход передаёт не «новый градиент u» и не весь готовый g_u, а один локальный вклад $\Delta g_u^{(v)}$ в полный градиент родителя. Полный g_u возникает только после накопления вкладов всех дочерних узлов u.

Результат шага. Мы различили данные, обучаемые параметры, вероятность, метку и среднюю потерю. Мы также ввели полный градиент g_v, локальный вклад $\Delta g_u^{(v)}$, направление прямых и обратных рёбер графа и увидели, почему полный градиент родителя складывается из вкладов всех его дочерних узлов. Далее построим логит одного объекта как скалярный DAG.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

2. Векторный вход как скалярный вычислительный граф

2.1. Аффинный узел и скалярное раскрытие

Сырые наблюдаемые данные сохраняют обозначение из раздела 1:

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id}).$$

Для аффинной операции к ним добавляется не новый наблюдаемый признак, а постоянная нулевая координата. Это **расширенный вход**:

$$\tilde{\mathbf{x}}_i = (\tilde{x}_{i0}, \tilde{x}_{i1}, \dots, \tilde{x}_{id}) = (1, x_{i1}, \dots, x_{id}).$$

Вектор параметров расширяется согласованно:

$$\tilde{\mathbf{w}} = (w_0, w_1, \dots, w_d).$$

В этой теории жирные векторы считаются векторами-столбцами. Верхний индекс T означает транспонирование: он превращает столбец в строку. Поэтому $\tilde{\mathbf{w}}^{\mathsf{T}}\tilde{\mathbf{x}}_i$ будет скаляром. До финального раздела главным источником смысла остаётся развёрнутая сумма: она в точности показывает скалярные вершины учебного DAG.

Нулевой вес w_0 называется **смещением**, или **весом постоянного признака**. Аффинный логит одного объекта теперь записывается одной универсальной суммой:

$$o_i = \sum_{\alpha=0}^{d} w_\alpha \tilde{x}_{i\alpha}.$$

При двух реальных признаках эта запись означает:

$$o_i = w_0 \cdot 1 + w_1 x_{i1} + w_2 x_{i2}.$$

В учебной функции `affine` расширенный список строится внутри функции как `[1.0, *features]`. Поэтому для двух признаков создаётся именно следующая последовательность скалярных операций:

$$q_{i0} = w_0 \tilde{x}_{i0} = w_0 \cdot 1, \qquad q_{i1} = w_1 x_{i1},$$

$$t_i = q_{i0} + q_{i1}, \qquad q_{i2} = w_2 x_{i2}, \qquad o_i = t_i + q_{i2}.$$

Это одна и та же аффинная функция, но развёрнутая в фактический порядок скалярных операций. В частности, $w_0 \cdot 1$ — настоящее умножение и настоящая вершина графа, а не присваивание ссылки на отдельное поле смещения.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

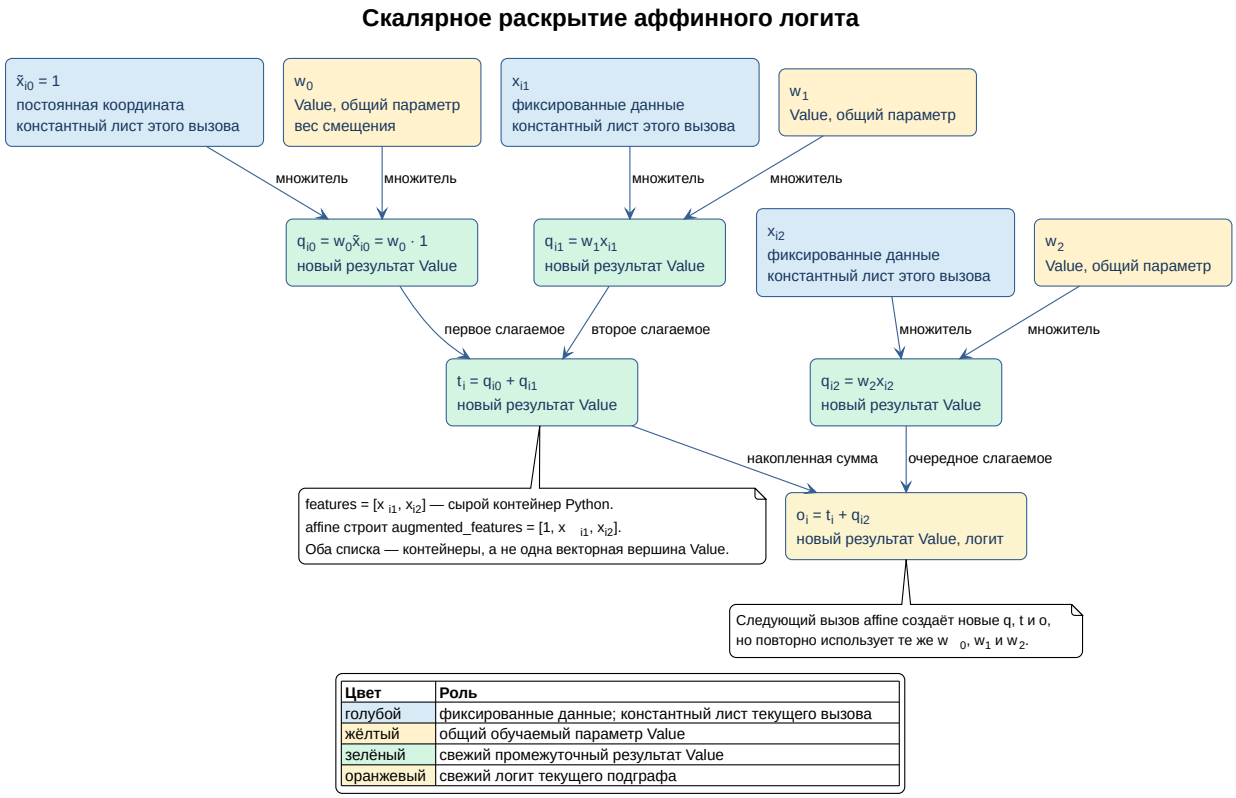


Рис. 4. Скалярный DAG аффинного выражения

В последующих схемах нейронов это же аффинное выражение будет показано одним укрупнённым блоком, чтобы рисунок оставался читаемым. Такой блок всегда означает раскрытую здесь последовательность скалярных умножений и сложений, а не новую операцию autograd.

2.2. Что является вершиной графа

Нужно различать сырой математический вектор, расширенный внутренний вектор, Python-контейнер и вершины скалярного DAG. Вектор $\mathbf{x}_i$ — запись наблюдаемых координат $x_{i1}, \ldots, x_{id}$. Список `features` содержит именно эти числа и служит контейнером Python. Внутри `affine` из него создаётся новый обычный список `augmented_features = [1.0, *features]`, соответствующий $\tilde{\mathbf{x}}_i$. Ни один из этих списков сам по себе не является одной вершиной `Value`.

В этой учебной реализации каждая координата x_{ij} сначала является обычным числом Python. Когда выражение `weight * feature` соединяет её с объектом `Value`, метод `__mul__` оборачивает число в константный лист текущего графа. Такой лист формально имеет `grad`, однако не является параметром и не обновляется. Роли величин удобно сопоставить так:

Математическая величина	Представление в Python	Роль в DAG одного вызова `affine`
$\mathbf{x}_i$	список `features`	сырой контейнер наблюдаемых признаков; не вершина `Value`
$\tilde{\mathbf{x}}_i$	`augmented_features = [1.0, *features]`	внутренний контейнер постоянной и реальных координат; не вершина `Value`

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Математическая величина	Представление в Python	Роль в DAG одного вызова `affine`
$\tilde{x}_{i0} = 1, x_{ij}$	обычные числа Python	константный скалярный лист после участия в операции
$w_0, w_1, \dots, w_d$	долгоживущие объекты `Value` в `weights`	общие обучаемые листья
q_{i0}, q_{ij}, t_i, o_i	новые результаты арифметических выражений	свежие промежуточные вершины

Поэтому длина `weights` должна быть на единицу больше длины `features`: нулевой вес w_0 соответствует постоянной координате 1, а каждый следующий вес соответствует ровно одной реальной координате. Проверка длины `weights` и `augmented_features` не даёт `zip` молча отбросить лишнюю координату или вес.

Следующий вызов `affine` для другого объекта создаёт новые константные листья, включая новую постоянную единицу, новые произведения, суммы и логит. Однако он присоединяет этот свежий подграф к тем же объектам $w_0, w_1, \dots, w_d$. Именно эта общность параметров станет причиной накопления вкладов по датасету в разделе 5.

2.3. Как скалярный autograd становится векторной моделью

Цикл не выполняет векторную операцию и не вызывает отдельный «векторный autograd». Он лишь многократно записывает обычные скалярные выражения `weight * feature` и `total + product`. В данной функции фактически вызываются уже реализованные методы `__mul__` и `__add__`; метод `__radd__` сохраняет тот же контракт скалярной арифметики для выражений, в которых обычное число находится слева от `Value`.

Каждая перегруженная операция создаёт результат `Value`, сохраняет ссылки на непосредственных родителей и локальное правило обратного прохода. Поэтому цикл динамически достраивает один ориентированный ациклический граф из скалярных узлов. Метод `backward()` и топологический порядок остаются теми же, что в скалярной лабораторной: им не нужно знать, что несколько однотипных путей возникли из координат вектора.

Здесь достаточно понять построение графа одного объекта. Детальное прохождение градиента через аффинный узел и накопление по всем объектам будут разобраны отдельно в разделе 5.

Результат шага. Векторность модели не создаёт векторную вершину и не требует нового класса autograd. Она означает повторение скалярных путей $w_\alpha \tilde{x}_{i\alpha}$ и сложений, в которых общие параметрические листья $w_0, w_1, \dots, w_d$ соединяются со свежими промежуточными вершинами конкретного объекта. Смещение больше не является особым полем: это нулевой вес постоянной координаты.

3. Один sigmoid-нейрон: прогноз, решение и функция потерь

Раздел 2 построил аффинный скалярный подграф для одного объекта. Теперь запишем ту же модель в общем виде для всего датасета. Для каждого объекта $i \in \{1, \dots, n\}$ сырой вход $\mathbf{x}_i$ превращается во внутренний расширенный вход $\tilde{\mathbf{x}}_i$. Индекс $j \in \{0, \dots, d\}$ фиксирует одну координату расширенного входа и соответствующий ей общий обучаемый вес. Немой индекс α перебирает все такие координаты внутри суммы. Логит объекта i имеет вид:

$$o_i = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i = \sum_{\alpha=0}^{d} w_\alpha \tilde{x}_{i\alpha}.$$

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

В частности, при $\alpha = 0$ в сумме участвуют постоянная координата $\tilde{x}_{i0} = 1$ и вес w_0. Запись со скалярным произведением компактно обозначает уже изученный скалярный DAG: для каждой координаты строится отдельное произведение $w_\alpha \tilde{x}_{i\alpha}$, а затем результаты складываются. Это не одна векторная вершина `Value`.

Логит o_i сам по себе не является вероятностью: он может быть любым вещественным числом. Для формального определения sigmoid введём s — произвольный скалярный аргумент, а не вектор:

$$\sigma : \mathbb{R} \to (0, 1), \qquad \sigma(s) = \frac{1}{1 + \exp(-s)}.$$

Для объекта i подставляем в эту скалярную функцию $s = o_i$ и получаем вероятность класса 1:

$$p_i = \sigma(o_i).$$

Sigmoid строго возрастает и отображает любое вещественное число в интервал $(0, 1)$. Поэтому порог $p_i \geq 0.5$ равносилен проверке $o_i \geq 0$. Чтобы описать геометрическую границу, рассмотрим произвольный возможный вход $\mathbf{x} = (x_1, \dots, x_d)$, а не фиксированный объект датасета $\mathbf{x}_i$. Для его расширенной записи $\tilde{\mathbf{x}} = (1, x_1, \dots, x_d)$ граница решения одного нейрона задаётся уравнением:

$$\sum_{\alpha=0}^{d} w_\alpha \tilde{x}_\alpha = 0.$$

Это аффинная гиперплоскость: при $d = 1$ она является точкой на прямой, при $d = 2$ — прямой на плоскости, а при больших d — её обобщением. В лабораторной $d = 2$, поэтому наглядная форма той же границы такова:

$$w_0 + w_1 x_1 + w_2 x_2 = 0.$$

Это одна прямая. Она может разделить классы OR и AND, но не может разделить классы XOR.

Потеря одного объекта

Вероятность должна не только порождать метку, но и сообщать, насколько хорош текущий прогноз. Для этого обучению нужна гладкая численная функция: она должна уменьшаться, когда модель повышает вероятность наблюдаемой правильной метки, и давать градиент параметрам.

Зафиксируем объект i. Величины o_i, p_i, y_i и будущая потеря ℓ_i — скаляры. Поэтому бинарная кросс-энтропия далее записывается для одного объекта, а индекс i сохраняет связь с его признаками и меткой. Это не отказ от векторной постановки: все объекты будут объединены в среднюю потерю L, а общие веса w_j получат вклады от каждого из них.

Бинарная кросс-энтропия (binary cross-entropy, BCE)

Для одного объекта модель назначает наблюдаемой метке y_i вероятность:

$$P(y_i \mid p_i) = p_i^{y_i} (1 - p_i)^{1 - y_i}.$$

Это единая запись двух случаев. При $y_i = 1$ она равна p_i: правильна метка класса 1. При $y_i = 0$ она равна $1 - p_i$: правильна метка класса 0.

При фиксированных параметрах модели считаем объекты датасета условно независимыми. Тогда правдоподобие датасета, то есть вероятность наблюдать все его метки по версии модели, равно произведению вероятностей правильных меток:

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

$$\mathcal{P} = \prod_{\mu=1}^{n} p_\mu^{y_\mu} (1 - p_\mu)^{1-y_\mu}.$$

Хорошая модель должна максимизировать $\mathcal{P}$. Логарифм строго возрастает, поэтому это равносильно максимизации $\log \mathcal{P}$; смена знака превращает задачу в минимизацию. Деление на n не меняет точки минимума, но делает масштаб потери независимым от размера датасета. Получаем:

$$L = -\frac{1}{n} \log \left(\prod_{\mu=1}^{n} p_\mu^{y_\mu} (1 - p_\mu)^{1-y_\mu} \right) = \frac{1}{n} \sum_{\mu=1}^{n} \left(-y_\mu \log(p_\mu) - (1 - y_\mu) \log(1 - p_\mu) \right).$$

Выражение в скобках — потеря одного объекта, называемая бинарной кросс-энтропией (англ. binary cross-entropy, BCE):

$$\ell_i = -y_i \log(p_i) - (1 - y_i) \log(1 - p_i).$$

Слово «бинарная» означает, что возможны два класса. Словосочетание «кросс-энтропия» здесь обозначает сравнение целевого распределения $(1 - y_i, y_i)$ с распределением, предсказанным моделью, $(1 - p_i, p_i)$. Для уже наблюдаемой конкретной метки BCE равна отрицательному логарифму вероятности правильного класса.

Если $y_i = 1$, второе слагаемое обнуляется и остаётся $-\log(p_i)$. Малое p_i тогда даёт большую потерю. Если $y_i = 0$, первое слагаемое обнуляется и остаётся $-\log(1 - p_i)$; большой прогноз вероятности класса 1 тоже получает большой штраф. При уверенном правильном прогнозе потеря стремится к 0; при уверенном неверном прогнозе она неограниченно возрастает.

Accuracy не заменяет BCE при обучении. Пороговое правило превращает оба прогноза $p_i = 0.51$ и $p_i = 0.99$ при $y_i = 0$ в одинаковую неверную метку 1. Между порогами accuracy не меняется, поэтому её производная по параметрам равна нулю, а в точке порога не определена. BCE различает степень ошибки и остаётся гладкой по $p_i \in (0, 1)$, поэтому `backward()` может использовать её градиент.

Потери всех объектов усредняются:

$$L = \frac{1}{n} \sum_{\mu=1}^{n} \ell_\mu.$$

Усреднение существенно: именно эта последняя скалярная величина будет аргументом `L.backward()` и источником градиентов всех общих параметров.

Как читать укрупнённые схемы нейронов

Начиная со следующего рисунка, схемы будут показывать аффинное выражение $o_i = \sum_{\alpha=0}^{d} w_\alpha \tilde{x}_{i\alpha}$ одним укрупнённым блоком. Этот блок не является ни одной атомарной вершиной `Value`, ни векторной операцией. Это краткое обозначение скалярного подграфа из подраздела 2.1: $q_{i0}, q_{i1}, t_i, q_{i2}, o_i$ при двух реальных признаках или его обобщения при d признаках.

Стрелка, входящая в аффинный блок или выходящая из него, на такой схеме обозначает зависимость всего этого подграфа, а не одну отдельную арифметическую операцию. Внутри блока сохраняются обычные вершины умножения и сложения, их родители и локальные правила reverse-mode. Поэтому обратный проход всё так же передаёт скалярные вклады; например, из логита к весу приходит $\Delta g_{w_j}^{(o_i)} = g_{o_i} \tilde{x}_{ij}$.

Итак, укрупнённая схема показывает уровень архитектуры нейрона, а не заменяет точный DAG реализации. В разделах 4 и 5 производные и обратный проход вновь будут раскрыты до скалярных операций.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

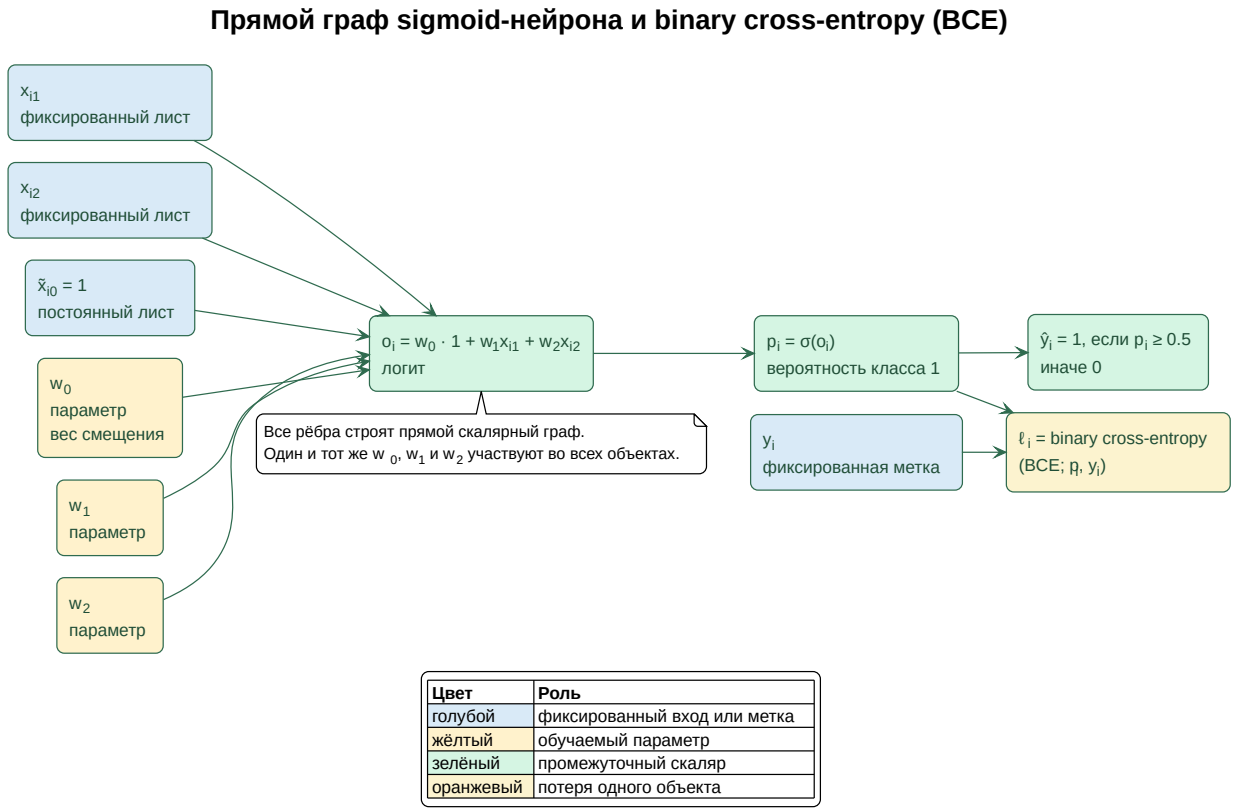


Рис. 5. Укрупнённая схема прямого графа одного sigmoid-нейрона

В коде sigmoid не добавляет специального метода в `Value`. Он раскладывается на уже реализованные скалярные операции:

```
(1.0 + (-logit).exp()) ** -1
```

Этот фрагмент создаёт вершины отрицания, экспоненты, сложения и степени. Функция `binary_cross_entropy` аналогично создаёт скалярный подграф из логарифмов, умножений, сложения и отрицания.

Результат шага. Полная прямая модель для каждого объекта имеет форму $\mathbf{x}_i \to \tilde{\mathbf{x}}_i \to o_i \to p_i \to \ell_i$, а средняя потеря объединяет все ℓ_i в L. Векторная запись управляет индексами объектов и координат, но каждый узел sigmoid и BCE остаётся скаляром. Теперь можно сначала вручную получить её производные, а затем увидеть те же числа в reverse-mode.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

4. Аналитическое дифференцирование одного sigmoid-нейрона

В этом разделе вычисляем производные так, как их выводят на бумаге. Рассматриваем фиксированный объект i: сырые признаки x_{ij}, постоянная координата $\tilde{x}_{i0} = 1$ и метка y_i постоянны, а веса $w_0, w_1, \dots, w_d$ независимы.

4.1. Sigmoid как композиция скалярных функций

Код sigmoid строит следующие промежуточные скаляры:

$$q_i = -o_i,$$

$$e_i = \exp(q_i),$$

$$d_i = 1 + e_i,$$

$$p_i = d_i^{-1}.$$

Локальные производные этих операций равны:

$$\frac{\partial p_i}{\partial d_i} = -d_i^{-2}, \qquad \frac{\partial d_i}{\partial e_i} = 1,$$

$$\frac{\partial e_i}{\partial q_i} = \exp(q_i) = e_i, \qquad \frac{\partial q_i}{\partial o_i} = -1.$$

Правило цепочки для всей композиции даёт:

$$\frac{\partial p_i}{\partial o_i} = \frac{\partial p_i}{\partial d_i}\frac{\partial d_i}{\partial e_i}\frac{\partial e_i}{\partial q_i}\frac{\partial q_i}{\partial o_i} = (-d_i^{-2}) \cdot 1 \cdot e_i \cdot (-1) = \frac{e_i}{d_i^2}.$$

Из $p_i = \frac{1}{d_i}$ и $d_i = 1 + e_i$ следует:

$$1 - p_i = 1 - \frac{1}{d_i} = \frac{d_i - 1}{d_i} = \frac{e_i}{d_i}.$$

Поэтому:

$$p_i(1 - p_i) = \frac{1}{d_i}\frac{e_i}{d_i} = \frac{e_i}{d_i^2}.$$

Следовательно:

$$\frac{\partial p_i}{\partial o_i} = p_i(1 - p_i).$$

Результат шага. Производная sigmoid получена из тех же четырёх скалярных узлов, которые строит `Value`. В следующем подшаге она будет умножена на производную ВСЕ по p_i.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

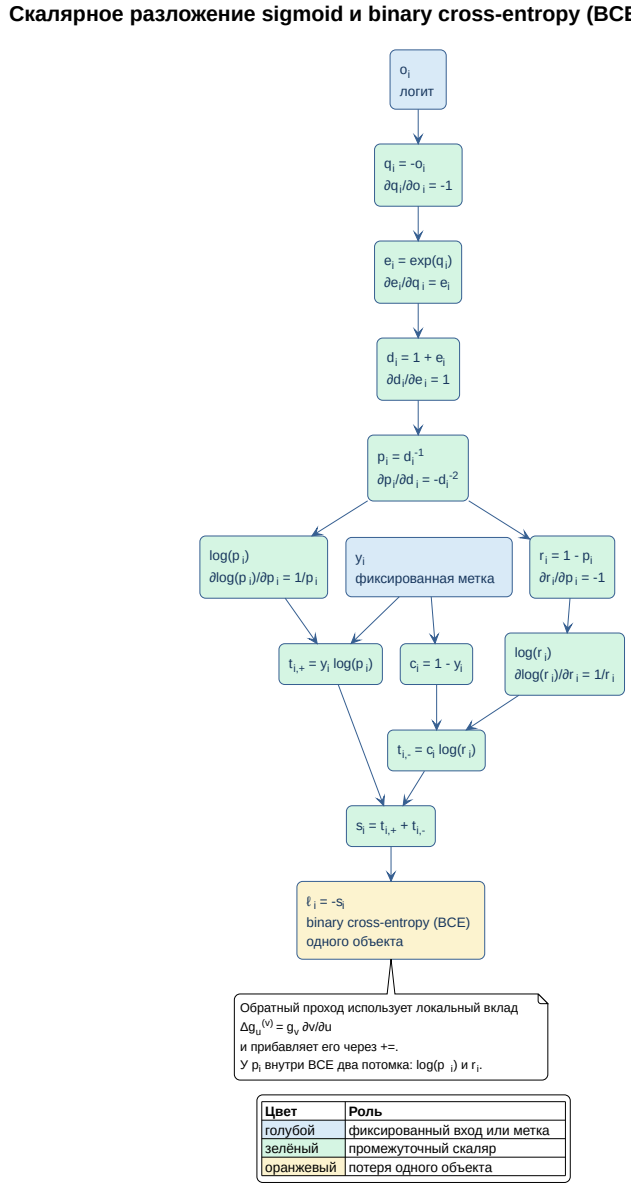


Рис. 6. Скалярное разложение sigmoid и BCE

4.2. Производная BCE по вероятности

Дифференцируем оба слагаемых ℓ_i по переменной p_i. В первом слагаемом y_i постоянно:

$$\frac{\partial\left(-y_i \log(p_i)\right)}{\partial p_i} = -y_i \frac{1}{p_i} = -\frac{y_i}{p_i}.$$

Во втором слагаемом есть внутренняя функция $1 - p_i$. Производная внешнего логарифма равна $\frac{1}{1-p_i}$, а производная внутренней функции равна -1. Поэтому два минуса дают плюс:

$$\frac{\partial\left(-(1-y_i)\log(1-p_i)\right)}{\partial p_i} = -(1-y_i)\frac{1}{1-p_i}(-1) = \frac{1-y_i}{1-p_i}.$$

Складываем оба результата:

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i}.$$

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

4.3. Локальная ошибка логита и средняя потеря

Теперь соединяем BCE и sigmoid:

$$\frac{\partial \ell_i}{\partial o_i} = \frac{\partial \ell_i}{\partial p_i}\frac{\partial p_i}{\partial o_i}.$$

Подстановка уже найденных выражений даёт:

$$\frac{\partial \ell_i}{\partial o_i} = \left(-\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i}\right) p_i(1-p_i),$$

$$\frac{\partial \ell_i}{\partial o_i} = -y_i(1-p_i) + (1-y_i)p_i,$$

$$\frac{\partial \ell_i}{\partial o_i} = -y_i + y_i p_i + p_i - y_i p_i,$$

$$\frac{\partial \ell_i}{\partial o_i} = p_i - y_i.$$

Введём короткое имя только для этой локальной производной одного объекта:

$$\delta_i = \frac{\partial \ell_i}{\partial o_i} = p_i - y_i.$$

Величина δ_i ещё не является градиентом средней потери L. Сначала усреднение передаёт:

$$g_{\ell_i} = \frac{\partial L}{\partial \ell_i} = \frac{1}{n}.$$

Поэтому полный входящий градиент логита равен:

$$g_{o_i} = \frac{\partial L}{\partial o_i} = g_{\ell_i}\delta_i = \frac{p_i - y_i}{n}.$$

4.4. Полные производные параметров

Для аффинного логита $o_i = \sum_{\alpha=0}^{d} w_\alpha \tilde{x}_{i\alpha}$ действует одно локальное правило для каждого фиксированного индекса $j = 0, \dots, d$:

$$\frac{\partial o_i}{\partial w_j} = \tilde{x}_{ij}.$$

Вклад одного объекта в общий градиент каждого параметра равен:

$$\Delta g_{w_j}^{(o_i)} = g_{o_i}\tilde{x}_{ij}, \qquad j = 0, \dots, d.$$

Один и тот же объект w_j участвует в графах всех объектов датасета. Поэтому полные производные являются суммами вкладов:

$$\frac{\partial L}{\partial w_j} = \sum_{\mu=1}^{n} g_{o_\mu}\tilde{x}_{\mu j} = \frac{1}{n}\sum_{\mu=1}^{n}(p_\mu - y_\mu)\tilde{x}_{\mu j}, \qquad j = 0, \dots, d.$$

Эти $d+1$ полных производных образуют один градиентный вектор-столбец:

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

$$\nabla_{\tilde{\mathbf{w}}} L = \begin{pmatrix} \frac{\partial L}{\partial w_0} \\ \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_d} \end{pmatrix} = \frac{1}{n} \sum_{\mu=1}^{n} (p_\mu - y_\mu) \begin{pmatrix} \tilde{x}_{\mu 0} \\ \tilde{x}_{\mu 1} \\ \vdots \\ \tilde{x}_{\mu d} \end{pmatrix}.$$

При $j = 0$ постоянная координата равна $\tilde{x}_{i0} = 1$. Первая компонента этого столбца, то есть формула нулевого веса, становится:

$$\frac{\partial L}{\partial w_0} = \frac{1}{n} \sum_{\mu=1}^{n} (p_\mu - y_\mu).$$

Следовательно, градиент смещения — это первая компонента $\nabla_{\tilde{\mathbf{w}}} L$. Он получен тем же правилом, что и градиенты остальных весов.

Результат шага. Ручной вывод закончен: $g_{o_i} = \frac{p_i - y_i}{n}$ приходит в аффинный узел, а сумма $g_{o_\mu} \tilde{x}_{\mu j}$ образует градиент каждого фиксированного веса w_j, включая w_0. Вектор $\nabla_{\tilde{\mathbf{w}}} L$ лишь собирает эти уже доказанные компоненты в одном столбце. Далее `Value` получит те же величины без записи этой готовой формулы в коде.

5. Reverse-mode получает те же градиенты

В прямом проходе `Value` сохраняет скалярные вершины, ссылки на их родителей, значения `data` и локальные функции `_backward`. В обратном проходе тот же DAG обходится в обратном топологическом порядке. Каждая локальная функция читает уже накопленный `result.grad`, умножает его на локальную производную и прибавляет вклад родителям.

5.1. Средняя потеря и seed-gradient

Сначала для всех объектов строятся o_i, p_i и ℓ_i. Их сумма и средняя потеря имеют вид:

$$S = \sum_{\mu=1}^{n} \ell_\mu, \qquad L = S \cdot \frac{1}{n}.$$

Метод `L.backward()` сначала обнуляет `grad` достижимых вершин и задаёт:

$$g_L = \frac{\partial L}{\partial L} = 1.$$

Умножение на $\frac{1}{n}$ передаёт в S значение $\frac{1}{n}$. Узлы сложения далее передают его каждой ℓ_i:

$$g_{\ell_i} = \frac{1}{n}.$$

Значит, до входа в BCE каждого объекта уже дошёл множитель усреднения. Его нельзя добавлять второй раз при накоплении весов.

5.2. BCE, sigmoid и аффинный узел одного объекта

Внутри `binary_cross_entropy` строятся скаляры:

$$r_i = 1 - p_i,$$

$$t_{i,+} = y_i \log(p_i), \qquad t_{i,-} = (1 - y_i) \log(r_i),$$

$$s_i = t_{i,+} + t_{i,-}, \qquad \ell_i = -s_i.$$

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Далее один объект проходит следующие четыре уровня обратного алгоритма.

1. Потеря сначала передаёт известный входящий градиент через отрицание и сумму. Из $\ell_i = -s_i$ следует:

$$g_{s_i} = g_{\ell_i}(-1).$$

Из $s_i = t_{i,+} + t_{i,-}$ следуют два вклада с коэффициентом 1:

$$g_{t_{i,+}} = g_{s_i}, \qquad g_{t_{i,-}} = g_{s_i}.$$

Затем множители y_i и $1 - y_i$ передают градиенты логарифмам:

$$g_{\log(p_i)} = g_{t_{i,+}} y_i, \qquad g_{\log(r_i)} = g_{t_{i,-}}(1 - y_i).$$

Здесь $g_{\ell_i} = 1/n$, потому что обратный проход уже прошёл усреднение. Метка y_i — константный лист: её собственный градиент для обучения не нужен.

2. Два пути ВСЕ сходятся в вершине p_i. Логарифм первого пути и вычитание во втором пути создают соответственно:

$$\Delta g_{p_i}^{(\log(p_i))} = g_{\log(p_i)} \frac{1}{p_i},$$

$$g_{r_i} = g_{\log(r_i)} \frac{1}{r_i}, \qquad \Delta g_{p_i}^{(r_i)} = g_{r_i}(-1).$$

Поэтому `p_i.grad` получает не замену, а сумму двух независимых вкладов:

$$g_{p_i} \leftarrow g_{p_i} + \Delta g_{p_i}^{(\log(p_i))} + \Delta g_{p_i}^{(r_i)}.$$

Именно это место показывает необходимость `+=`: один и тот же узел p_i одновременно участвует в $\log(p_i)$ и в $\log(1 - p_i)$.

3. Sigmoid передаёт уже накопленный g_{p_i} по сохраненной цепочке $p_i \to d_i \to e_i \to q_i \to o_i$:

$$\Delta g_{d_i}^{(p_i)} = g_{p_i}(-d_i^{-2}),$$

$$\Delta g_{e_i}^{(d_i)} = g_{d_i}, \qquad \Delta g_{q_i}^{(e_i)} = g_{e_i} e_i, \qquad \Delta g_{o_i}^{(q_i)} = g_{q_i}(-1).$$

Их произведение совпадает с аналитическим результатом:

$$g_{o_i} = g_{p_i} p_i(1 - p_i) = \frac{p_i - y_i}{n}.$$

4. Наконец аффинный узел передаёт градиент параметрам и координатам расширенного входа:

$$\Delta g_{w_j}^{(o_i)} = g_{o_i} \tilde{x}_{ij}, \qquad \Delta g_{\tilde{x}_{ij}}^{(o_i)} = g_{o_i} w_j, \qquad j = 0, \dots, d.$$

При $j = 0$ получаем $\Delta g_{w_0}^{(o_i)} = g_{o_i} \cdot 1$. Последняя формула описывает реальный путь к постоянной или к признаку. Однако $\tilde{x}_{i0} = 1$ и x_{ij} — константные листья, поэтому они не обновляются.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

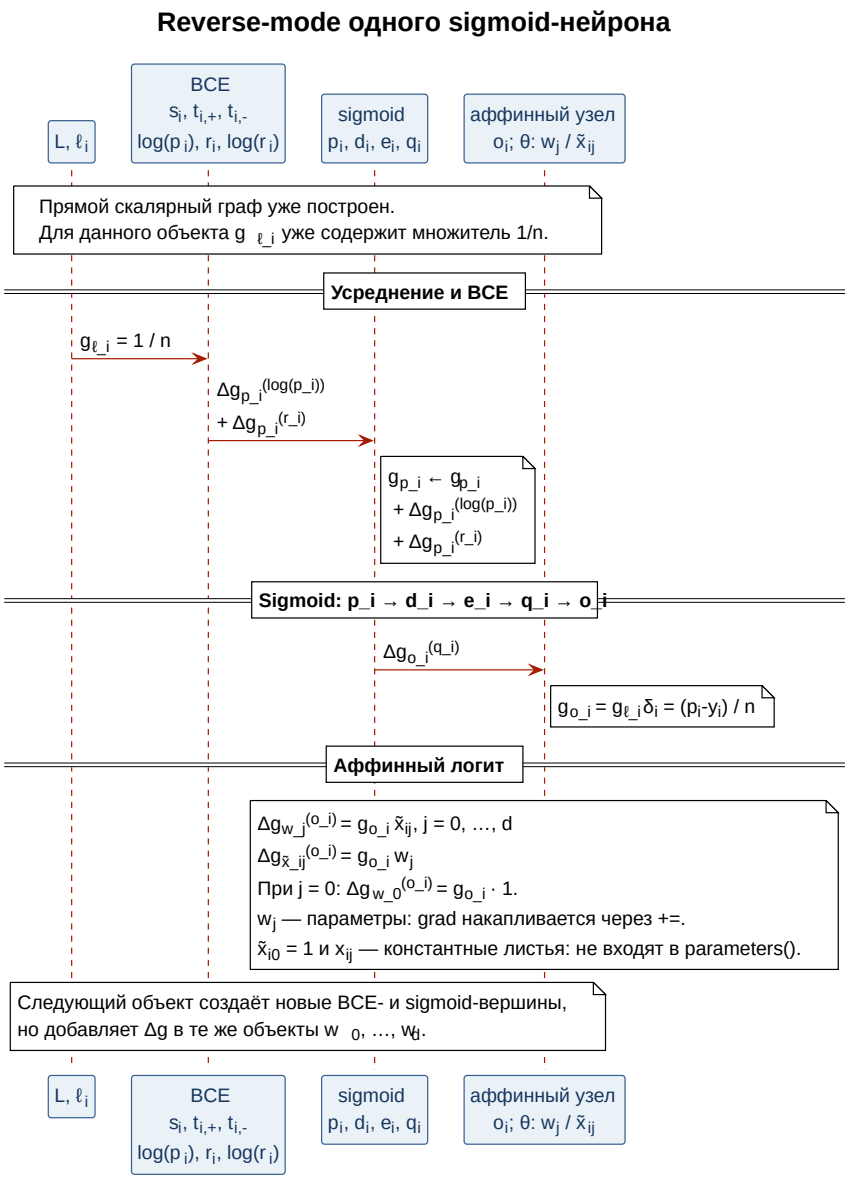


Рис. 7. Строгая последовательность reverse-mode одного sigmoid-нейрона

5.3. Общие параметры всего датасета

Каждый объект создаёт новые вершины $q_i, e_i, d_i, p_i, \ell_i, o_i$. Веса $w_0, w_1, \dots, w_d$ остаются теми же объектами `Value`. Поэтому после обработки очередного объекта для каждого $j = 0, \dots, d$ выполняется:

$$g_{w_j} \leftarrow g_{w_j} + \Delta g_{w_j}^{(o_i)}.$$

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Два объекта используют общие параметры и суммируют градиенты

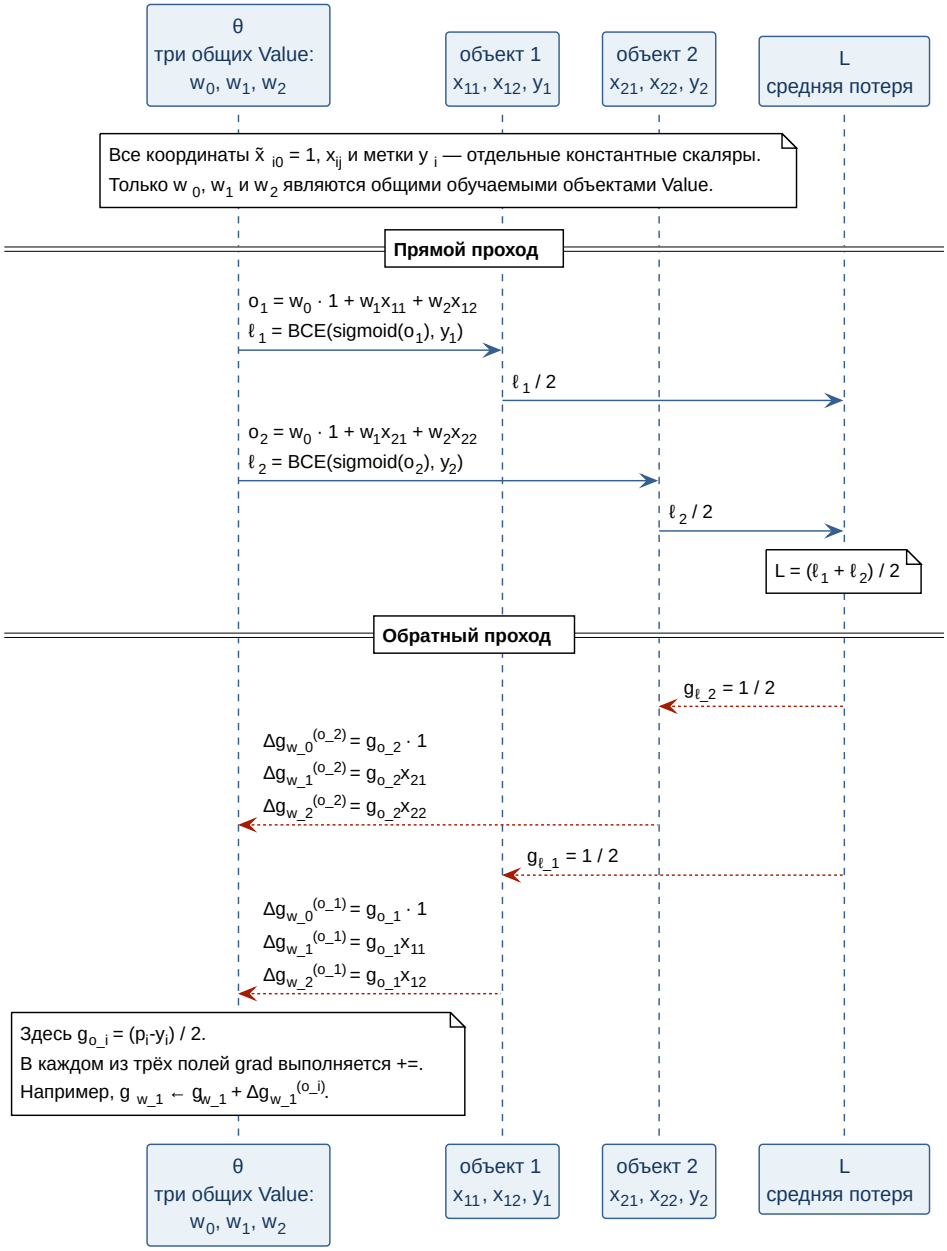


Рис. 8. Общие параметры и накопление градиентов датасета

Результат шага. Аналитическое дифференцирование и reverse-mode вычисляют одни и те же производные. Разница только в организации вычисления: ручной вывод сокращает формулу, а `Value` проходит сохранённый DAG и накапливает локальные вклады.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

6. Один скрытый нейрон: композиция и цепное правило на AND

AND линейно разделима, поэтому один sigmoid-нейрон уже способен её выучить. Скрытый нейрон вводится здесь не для увеличения выразительности на AND, а как минимальный наблюдаемый пример композиции двух обучаемых преобразований: градиент должен пройти через выходную аффинную операцию, нелинейность tanh и скрытую аффинную операцию. Так следующий шаг отделяет вопрос о механизме обучения от будущего вопроса о выразительности архитектуры.

Следующая модель содержит скрытый tanh-нейрон и sigmoid-выход. Для скрытого аффинного узла используем расширенный вектор весов:

$$\tilde{\mathbf{u}} = (u_0, u_1, u_2), \qquad r_i = \sum_{\alpha=0}^{2} u_\alpha \tilde{x}_{i\alpha} = u_0 + u_1 x_{i1} + u_2 x_{i2}.$$

$$h_i = \tanh(r_i).$$

Для выходной аффинной операции расширяем скрытый вход:

$$\tilde{\mathbf{h}}_i = (\tilde{h}_{i0}, \tilde{h}_{i1}) = (1, h_i), \qquad \tilde{\mathbf{v}} = (v_0, v_1),$$

$$o_i = \sum_{\beta=0}^{1} v_\beta \tilde{h}_{i\beta} = v_0 + v_1 h_i, \qquad p_i = \sigma(o_i).$$

Здесь u_0 — смещение скрытого нейрона, а v_0 — смещение выходного нейрона: оба являются нулевыми весами постоянных координат. Гладкая функция tanh помещает скрытое значение в интервал $(-1, 1)$, а sigmoid на выходе по-прежнему даёт вероятность бинарного класса.

Следующая укрупнённая схема использует соглашение раздела 3: каждый аффинный блок разворачивается в скалярный DAG по правилу подраздела 2.1.

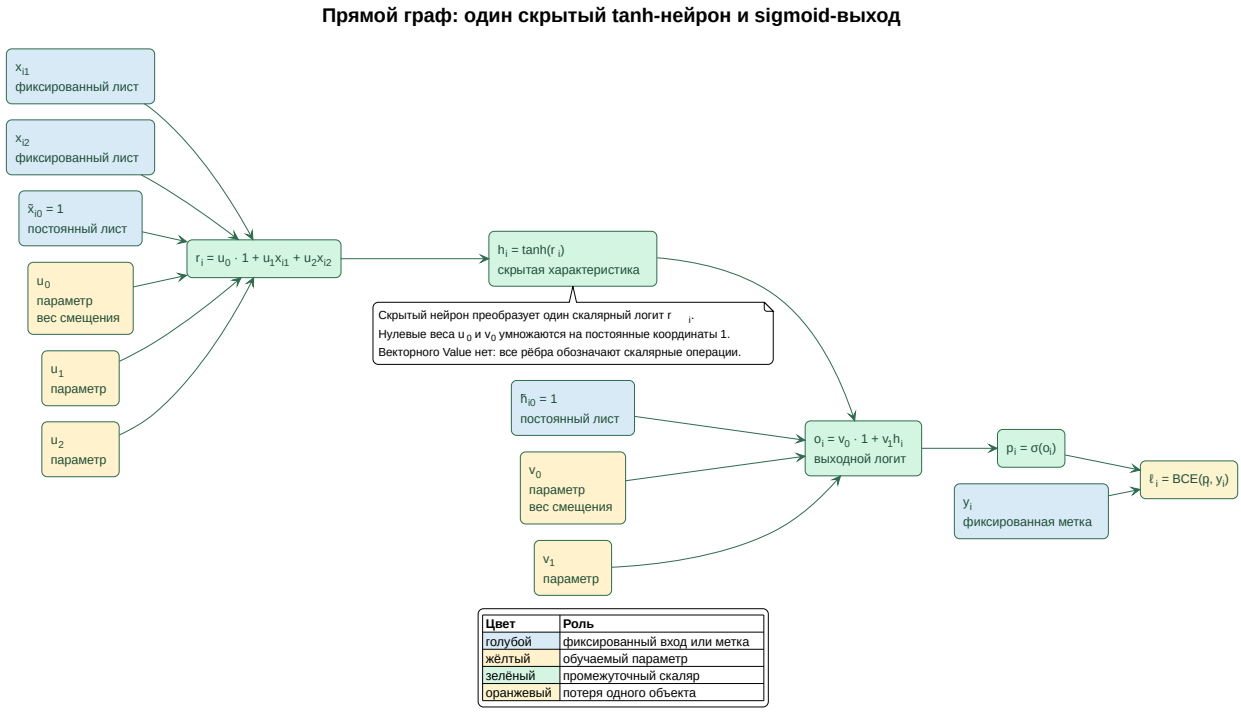


Рис. 9. Укрупнённая схема прямого графа одного скрытого и одного выходного нейрона

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Из предыдущих разделов в выходной логит уже приходит:

$$g_{o_i} = \frac{p_i - y_i}{n}.$$

Выходная аффинная операция передаёт для $k = 0, 1$:

$$\Delta g_{v_k}^{(o_i)} = g_{o_i} \tilde{h}_{ik}.$$

В частности, $\Delta g_{v_0}^{(o_i)} = g_{o_i} \cdot 1$, а путь к скрытому состоянию использует вес v_1:

$$g_{h_i} = g_{o_i} v_1.$$

У скрытого значения h_i в этой модели один потомок, поэтому его накопленный градиент действительно равен показанному вкладу. Tanh передаёт его к скрытому логиту:

$$g_{r_i} = g_{h_i}(1 - h_i^2) = g_{o_i} v_1 (1 - h_i^2).$$

Скрытая аффинная операция добавляет для $j = 0, 1, 2$:

$$\Delta g_{u_j}^{(r_i)} = g_{r_i} \tilde{x}_{ij}.$$

В частности, $\Delta g_{u_0}^{(r_i)} = g_{r_i} \cdot 1$. Нулевые веса обеих аффинных операций участвуют в обратном проходе по общему правилу, а не по отдельному правилу для смещения.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

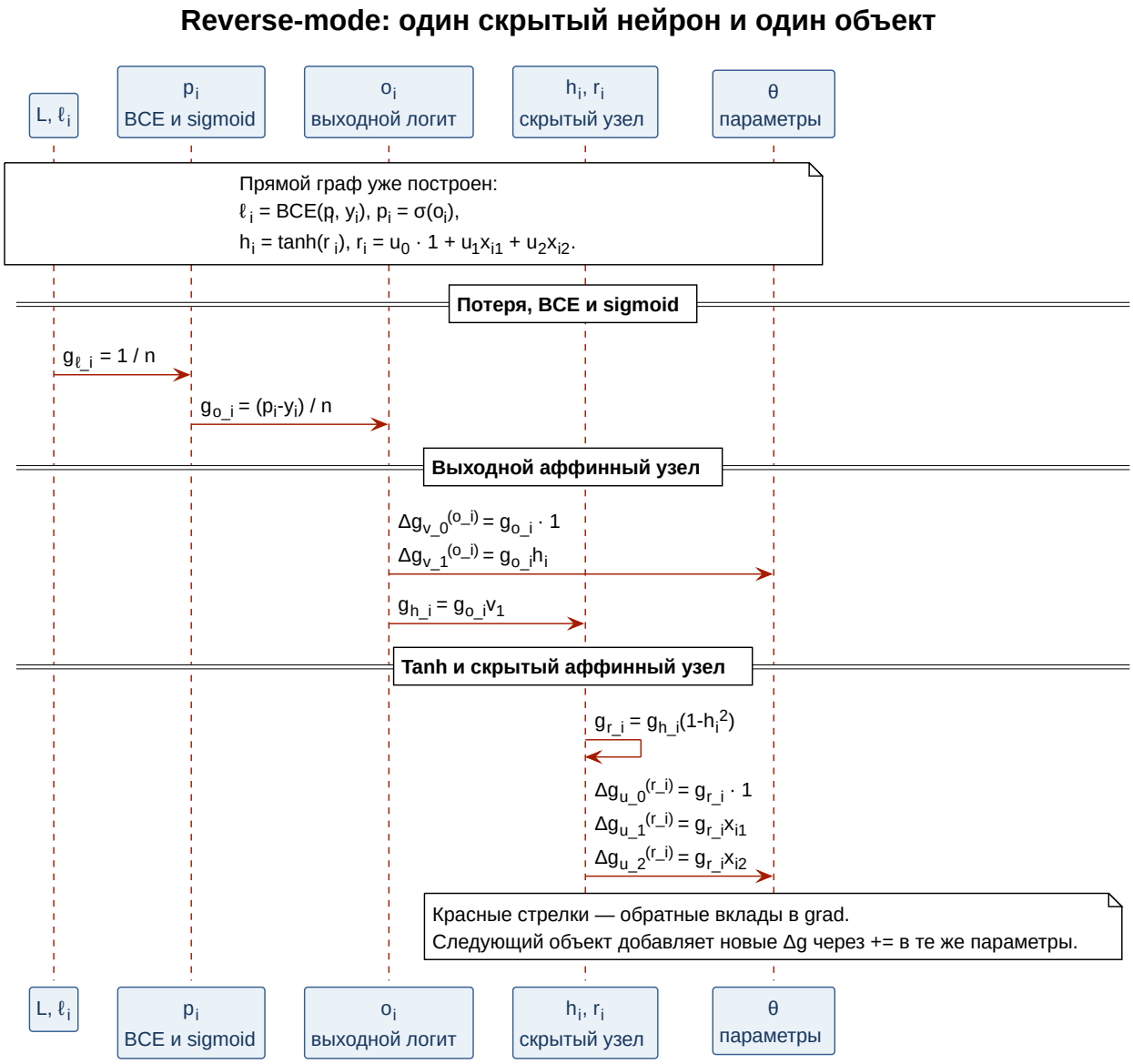


Рис. 10. Строгая последовательность обратного прохода одного объекта

Результат шага. Величина g_{o_i} проходит через два уровня параметров. Это тот же принцип локальных производных из скалярной лабораторной, но теперь результат одного уровня становится входящим градиентом следующего.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

7. Зачем в маршруте нужен XOR: ограничение одного скрытого нейрона

OR показал работу одной аффинной границы, а AND позволил проследить градиент через композицию. Теперь меняется вопрос: не как обучается модель, а какие зависимости она вообще способна представить. Для этого берём XOR из таблицы подраздела 1: точки $(0, 1)$ и $(1, 0)$ имеют метку 1, а точки $(0, 0)$ и $(1, 1)$ имеют метку 0.

Порог $p_i \geq 0.5$ эквивалентен $o_i \geq 0$. Рассмотрим нетривиальный случай $v_1 > 0$ и $-v_0/v_1 \in (-1, 1)$. Тогда:

$$o_i \geq 0 \quad \Longleftrightarrow \quad \tanh(r_i) \geq -\frac{v_0}{v_1}.$$

Так как tanh строго возрастает, применяем обратную функцию arctanh:

$$u_0 + u_1 x_{i1} + u_2 x_{i2} \geq \operatorname{arctanh}\left(-\frac{v_0}{v_1}\right).$$

Это неравенство задаёт полуплоскость. При $v_1 < 0$ знак неравенства меняется, но граница остаётся прямой. При $v_1 = 0$ или при $-v_0/v_1$ вне интервала $(-1, 1)$ прогноз постоянен для всех конечных r_i.

Остаётся показать, почему одна прямая не способна разделить XOR. Пусть:

$$f(x_1, x_2) = c_0 + c_1 x_1 + c_2 x_2.$$

Для четырёх углов выполняется точное равенство:

$$f(0, 1) + f(1, 0) == c_1 + c_2 + 2c_0 = f(0, 0) + f(1, 1).$$

Для точной классификации XOR левая сумма должна быть неотрицательной, поскольку обе точки принадлежат классу 1. Правая сумма должна быть строго отрицательной, поскольку обе точки принадлежат классу 0. Одно и то же число не может одновременно удовлетворять этим двум условиям.

Результат шага. Один скрытый нейрон после любого монотонного порога всё ещё задаёт одну полуплоскость. Чтобы представить XOR, нужны хотя бы две независимые нелинейные скрытые характеристики.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

8. Два скрытых нейрона: XOR как минимальный тест выразительности

Введём два скрытых нейрона с индексом $h \in \{1, 2\}$. У каждого скрытого нейрона есть расширенный вектор весов:

$$\tilde{\mathbf{u}}_h = (u_{h0}, u_{h1}, u_{h2}), \qquad r_{ih} = \sum_{\alpha=0}^{2} u_{h\alpha}\tilde{x}_{i\alpha} = u_{h0} + u_{h1}x_{i1} + u_{h2}x_{i2}.$$

$$h_{ih} = \tanh(r_{ih}).$$

Выходной нейрон получает расширенный скрытый вход:

$$\tilde{\mathbf{h}}_i = (\tilde{h}_{i0}, \tilde{h}_{i1}, \tilde{h}_{i2}) = (1, h_{i1}, h_{i2}),$$

$$\tilde{\mathbf{v}} = (v_0, v_1, v_2), \qquad o_i = \sum_{\beta=0}^{2} v_\beta \tilde{h}_{i\beta} = v_0 + v_1 h_{i1} + v_2 h_{i2}, \qquad p_i = \sigma(o_i).$$

Два значения h_{i1} и h_{i2} являются независимыми нелинейными характеристиками одного входного объекта. Выходная аффинная операция комбинирует их. Благодаря этому граница классификации больше не обязана быть одной прямой в исходной плоскости признаков.

Здесь сохраняется соглашение раздела 3: каждый аффинный блок на схеме — компактная запись скалярного DAG из подраздела 2.1, а не отдельная векторная вершина.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

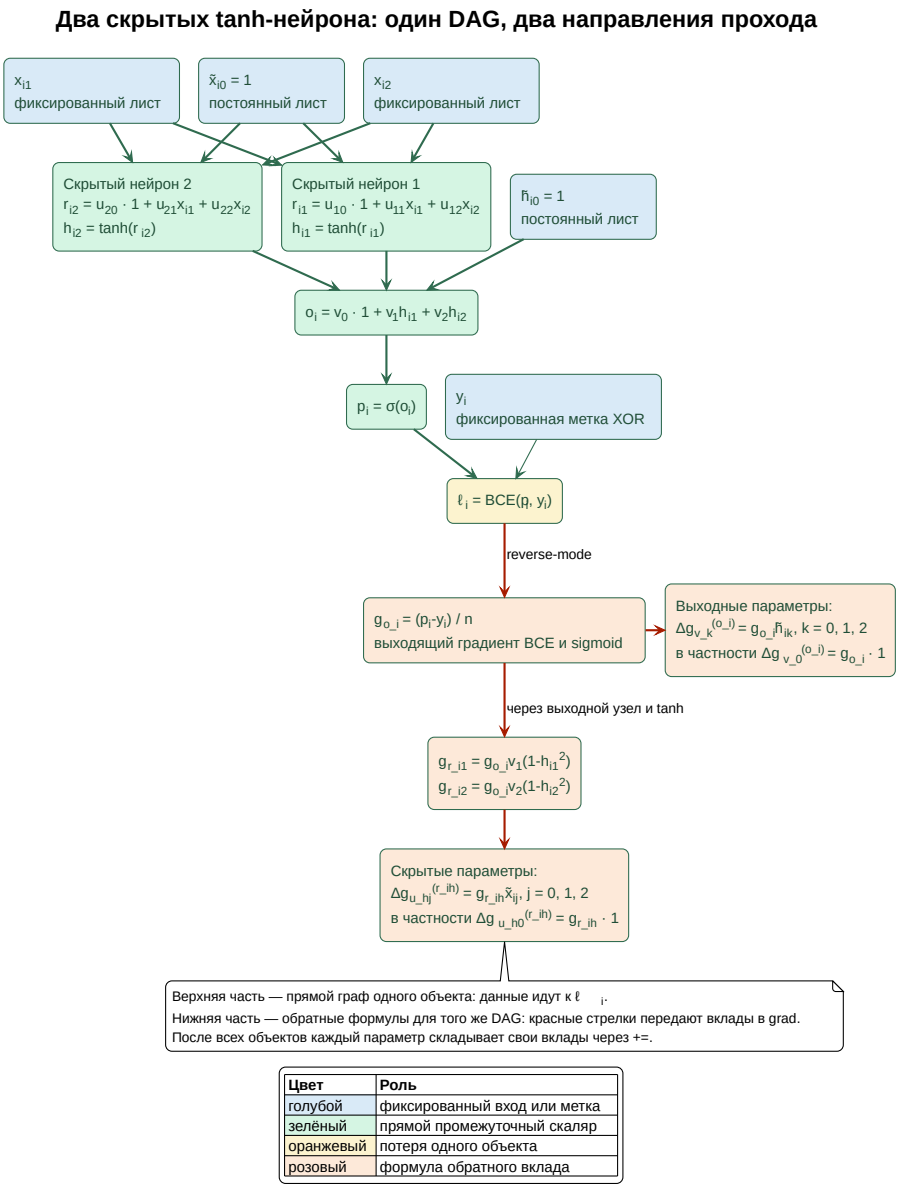


Рис. 11. Укрупнённая схема прямого и обратного проходов слоя из двух скрытых нейронов

8.1. Обратный проход скрытого слоя

Из BCE и sigmoid вновь получаем:

$$g_{o_i} = \frac{p_i - y_i}{n}.$$

Для каждого скрытого нейрона h выходной аффинный узел передаёт:

$$g_{h_{ih}} = g_{o_i} v_h.$$

Tanh передаёт полный градиент к своему логиту:

$$g_{r_{ih}} = g_{h_{ih}}(1 - h_{ih}^2) = g_{o_i} v_h (1 - h_{ih}^2).$$

Здесь намеренно используется $g_{r_{ih}}$, а не новое обозначение δ_{ih}. Величина уже включает множитель $\frac{1}{n}$ через g_{o_i}, то есть является градиентом средней потери L, а не локальной производной одной потери ℓ_i.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Один объект добавляет в параметры скрытого слоя:

$$\Delta g_{u_{hj}}^{(r_{ih})} = g_{r_{ih}} \tilde{x}_{ij}, \qquad j = 0, 1, 2.$$

Он же добавляет в параметры выходного слоя:

$$\Delta g_{v_k}^{(o_i)} = g_{o_i} \tilde{h}_{ik}, \qquad k = 0, 1, 2.$$

После суммирования объектов получаем полные производные:

$$\frac{\partial L}{\partial u_{hj}} = \sum_{\mu=1}^{n} g_{r_{\mu h}} \tilde{x}_{\mu j}, \qquad j = 0, 1, 2,$$

$$\frac{\partial L}{\partial v_k} = \sum_{\mu=1}^{n} g_{o_\mu} \tilde{h}_{\mu k}, \qquad k = 0, 1, 2.$$

Эти покомпонентные формулы можно свернуть в одну запись для одного объекта. Соберём веса скрытого слоя, его логиты и состояния в столбцы:

$$\mathbf{U} = \begin{pmatrix} u_{10} & u_{11} & u_{12} \\ u_{20} & u_{21} & u_{22} \end{pmatrix}, \qquad \mathbf{r}_i = \mathbf{U}\tilde{\mathbf{x}}_i, \qquad \mathbf{h}_i = \tanh(\mathbf{r}_i) = \begin{pmatrix} h_{i1} \\ h_{i2} \end{pmatrix}.$$

В этой компактной записи tanh применяется к двум компонентам по отдельности. Обозначение $\odot$ далее означает поэлементное умножение векторов одинаковой длины, а $\mathbf{1}$ — столбец единиц нужной длины. Тогда два значения $g_{r_{i1}}$ и $g_{r_{i2}}$ образуют столбец:

$$\mathbf{g}_{\mathbf{r}_i} = g_{o_i} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \odot (\mathbf{1} - \mathbf{h}_i \odot \mathbf{h}_i).$$

Вклады объекта i в параметры записываются как вектор и внешнее произведение:

$$\Delta \mathbf{G}_{\mathbf{U}}^{(i)} = \mathbf{g}_{\mathbf{r}_i} \tilde{\mathbf{x}}_i^{\mathsf{T}}, \qquad \Delta \mathbf{g}_{\tilde{\mathbf{v}}}^{(i)} = g_{o_i} \tilde{\mathbf{h}}_i.$$

Элемент строки h и столбца j матрицы $\Delta \mathbf{G}_{\mathbf{U}}^{(i)}$ равен $g_{r_{ih}} \tilde{x}_{ij}$, то есть уже доказанному $\Delta g_{u_{hj}}^{(r_{ih})}$. Компонента k столбца $\Delta \mathbf{g}_{\tilde{\mathbf{v}}}^{(i)}$ равна $g_{o_i} \tilde{h}_{ik}$, то есть $\Delta g_{v_k}^{(o_i)}$. Новая запись ничего не меняет в обратном проходе: она только группирует прежние скалярные вклады.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

8.2. Итоговый цикл обучения

Одна эпоха объединяет весь материал лекции:

```
loss = mean_binary_cross_entropy(model, inputs, targets)
loss.backward()

for parameter in model.parameters():
    parameter.data -= learning_rate * parameter.grad
```

Первая строка строит свежий DAG всех объектов и итоговую среднюю потерю L. Вторая запускает обратный топологический обход, который заполняет `grad` общих параметров. Цикл обновления выполняется только после `backward()`: изменение `parameter.data` до обратного прохода сделало бы сохранённые значения графа несогласованными с прямым проходом.

Формулы, вершины графа, действия программы и результаты лабораторной образуют одну систему:

Формула или правило	Вершина графа	Действие в программе	Проверяемый результат
$o_i = \sum_{\alpha=0}^{d} w_\alpha \tilde{x}_{i\alpha}$	аффинный логит	`affine`	один sigmoid-нейрон обучает OR
$\ell_i = -y_i \log p_i - (1-y_i)\log(1-p_i)$	BCE	`binary_cross_entropy`	loss измеряет качество вероятности
$g_{o_i} = \frac{p_i - y_i}{n}$	путь BCE и sigmoid	`loss.backward()`	градиент проходит к весам, включая нулевой вес
$g_{r_i} = g_{o_i} v_1 (1 - h_i^2)$	скрытый tanh-узел	`model(x)`	AND демонстрирует цепное правило через скрытый узел
$g_{r_{ih}} = g_{o_i} v_h (1 - h_{ih}^2)$	два скрытых узла	`model(x)`	XOR достигает accuracy 1.0

В notebook зафиксированы итоговые диагностические критерии:

Эксперимент	Проверка качества
OR, один sigmoid-нейрон	BCE < 0.1, accuracy $= 1.0$
AND, один скрытый нейрон	BCE < 0.05, accuracy $= 1.0$
XOR, два скрытых нейрона	BCE < 0.03, accuracy $= 1.0$

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

9. От скалярных формул к векторам и матрицам

Итог. Ранее мы намеренно раскрывали каждую координату, вершину и локальный вклад. Теперь те же результаты можно записать короче. Эта запись не заменяет уже доказанные формулы: она группирует их и станет удобной, когда число объектов, признаков и скрытых нейронов возрастёт.

9.1. Матрица датасета и соглашение о формах

Соберём расширенные входы объектов в матрицу строками. Каждая строка — это транспонированный расширенный вход одного объекта:

$$\tilde{\mathbf{X}} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1d} \\ 1 & x_{21} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{nd} \end{pmatrix} \in \mathbb{R}^{n\times(d+1)}, \qquad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^{n\times 1}.$$

Векторы результатов всех объектов также будут столбцами. Для столбца $\mathbf{a} = (a_1, \dots, a_n)^\mathsf{T}$ определим покоординатное применение функций:

$$\sigma(\mathbf{a}) = \begin{pmatrix} \sigma(a_1) \\ \vdots \\ \sigma(a_n) \end{pmatrix}, \qquad \log(\mathbf{a}) = \begin{pmatrix} \log(a_1) \\ \vdots \\ \log(a_n) \end{pmatrix}.$$

Если $\mathbf{a}$ и $\mathbf{b}$ имеют одинаковую форму, то $\mathbf{a} \odot \mathbf{b}$ обозначает поэлементное умножение. Столбец $\mathbf{1}_n$ содержит n единиц. Эти соглашения используются только в этом итоговом обобщении; в разделах 1–8 sigmoid, логарифм и BCE были скалярными операциями конкретных вершин графа.

9.2. Один sigmoid-нейрон как матричная формула

Параметры одного нейрона образуют столбец $\tilde{\mathbf{w}} \in \mathbb{R}^{(d+1)\times 1}$. Его логиты, вероятности и вектор локальных ошибок всех объектов равны:

$$\mathbf{o} = \tilde{\mathbf{X}}\tilde{\mathbf{w}}, \qquad \mathbf{p} = \sigma(\mathbf{o}), \qquad \delta = \mathbf{p} - \mathbf{y}.$$

Вектор δ содержит уже полученные в разделе 4 величины $\delta_i = p_i - y_i$. Средняя BCE в матричной записи имеет вид:

$$L = -\frac{1}{n}\mathbf{1}_n^\mathsf{T}\left(\mathbf{y} \odot \log(\mathbf{p}) + (\mathbf{1}_n - \mathbf{y}) \odot \log(\mathbf{1}_n - \mathbf{p})\right).$$

Поэтому полный градиент логитов и градиент всех параметров сразу равны:

$$\mathbf{g}_\mathbf{o} = \nabla_\mathbf{o} L = \frac{1}{n}(\mathbf{p} - \mathbf{y}), \qquad \nabla_{\tilde{\mathbf{w}}} L = \tilde{\mathbf{X}}^\mathsf{T}\mathbf{g}_\mathbf{o} = \frac{1}{n}\tilde{\mathbf{X}}^\mathsf{T}\delta.$$

Компонента с индексом j столбца $\nabla_{\tilde{\mathbf{w}}} L$ равна $\frac{1}{n}\sum_{\mu=1}^{n}(p_\mu - y_\mu)\tilde{x}_{\mu j}$. Тем самым матричное произведение точно совпадает с покомпонентной формулой раздела 4, включая $j = 0$ и градиент смещения.

9.3. Скрытый слой из m tanh-нейронов

Пусть скрытый слой содержит m нейронов. Строка h матрицы $\mathbf{U}$ содержит расширенный вектор весов скрытого нейрона h:

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

$$\mathbf{U} = \begin{pmatrix} u_{10} & u_{11} & \cdots & u_{1d} \\ u_{20} & u_{21} & \cdots & u_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ u_{m0} & u_{m1} & \cdots & u_{md} \end{pmatrix} \in \mathbb{R}^{m\times(d+1)}.$$

Прямой проход всего датасета записывается так:

$$\mathbf{R} = \tilde{\mathbf{X}}\mathbf{U}^{\mathsf{T}} \in \mathbb{R}^{n\times m}, \qquad \mathbf{H} = \tanh(\mathbf{R}) \in \mathbb{R}^{n\times m}.$$

Здесь tanh применяется к каждому элементу матрицы $\mathbf{R}$. Добавим к каждой строке скрытых состояний постоянную единицу и зададим веса выходного нейрона:

$$\tilde{\mathbf{H}} = \begin{pmatrix} 1 & h_{11} & \cdots & h_{1m} \\ 1 & h_{21} & \cdots & h_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & h_{n1} & \cdots & h_{nm} \end{pmatrix} \in \mathbb{R}^{n\times(m+1)}, \qquad \tilde{\mathbf{v}} = \begin{pmatrix} v_0 \\ v_1 \\ \vdots \\ v_m \end{pmatrix}.$$

Тогда:

$$\mathbf{o} = \tilde{\mathbf{H}}\tilde{\mathbf{v}}, \qquad \mathbf{p} = \sigma(\mathbf{o}), \qquad \mathbf{g_o} = \frac{1}{n}(\mathbf{p} - \mathbf{y}).$$

Введём столбец выходных весов без веса постоянной координаты:

$$\mathbf{v}_{1:m} = \begin{pmatrix} v_1 \\ \vdots \\ v_m \end{pmatrix}.$$

Матрица полных градиентов скрытых логитов, а затем матрица и вектор градиентов параметров имеют вид:

$$\mathbf{G_R} = \nabla_{\mathbf{R}} L = (\mathbf{g_o}\mathbf{v}_{1:m}^{\mathsf{T}}) \odot (\mathbf{1}_{n\times m} - \mathbf{H}\odot\mathbf{H}),$$

$$\nabla_{\tilde{\mathbf{v}}} L = \tilde{\mathbf{H}}^{\mathsf{T}}\mathbf{g_o}, \qquad \nabla_{\mathbf{U}} L = \mathbf{G_R}^{\mathsf{T}}\tilde{\mathbf{X}}.$$

Элемент (h, j) матрицы $\nabla_{\mathbf{U}} L$ равен $\sum_{\mu=1}^{n} g_{r_{\mu h}}\tilde{x}_{\mu j}$. Компонента k вектора $\nabla_{\tilde{\mathbf{v}}} L$ равна $\sum_{\mu=1}^{n} g_{o_\mu}\tilde{h}_{\mu k}$. Это в точности формулы раздела 8, записанные без явного цикла по объектам.

При $m = 1$ получаем модель AND из раздела 6: $\mathbf{U}$ имеет одну строку, а $\tilde{\mathbf{v}}$ состоит из v_0 и v_1. При $m = 2$ получаем XOR-модель раздела 8: у $\mathbf{U}$ две строки, а $\tilde{\mathbf{v}}$ состоит из v_0, v_1 и v_2.

9.4. Соответствие трёх записей

Матричные формулы лишь группируют скалярные операции `Value`, не меняя учебный autograd; так XOR-модель получает точную векторно-матричную запись. Практическая реализация остаётся скалярной — переходите к выданной пошаговой лабораторной работе.

Покомпонентная формула	Векторно-матричная запись	Скалярный DAG и программа
$o_i = \sum_{\alpha=0}^{d} w_\alpha \tilde{x}_{i\alpha}$	$\mathbf{o} = \tilde{\mathbf{X}}\tilde{\mathbf{w}}$	`affine`: скалярные умножения и суммы
$\frac{\partial L}{\partial w_j} = \sum_{\mu=1}^{n} g_{o_\mu}\tilde{x}_{\mu j}$	$\nabla_{\tilde{\mathbf{w}}} L = \tilde{\mathbf{X}}^{\mathsf{T}}\mathbf{g_o}$	общий вес накапливает `grad`
$\frac{\partial L}{\partial u_{hj}} = \sum_{\mu=1}^{n} g_{r_{\mu h}}\tilde{x}_{\mu j}$	$\nabla_{\mathbf{U}} L = \mathbf{G_R}^{\mathsf{T}}\tilde{\mathbf{X}}$	вклад проходит к скрытому весу
$\frac{\partial L}{\partial v_k} = \sum_{\mu=1}^{n} g_{o_\mu}\tilde{h}_{\mu k}$	$\nabla_{\tilde{\mathbf{v}}} L = \tilde{\mathbf{H}}^{\mathsf{T}}\mathbf{g_o}$	выходной вес общий для объектов

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

Необязательный мостик: нотация Эйнштейна

Иногда в формулах линейной алгебры знак суммы опускают: повторяющийся немой индекс подразумевает свёртку. Для аффинного логита одна и та же запись в двух соглашениях имеет вид:

$$o_i = \sum_{\alpha=0}^{d} w_\alpha \tilde{x}_{i\alpha},$$

$$o_i = w_\alpha \tilde{x}_{i\alpha}.$$

Во второй строке повторение только α означает суммирование от 0 до d; свободный i сохраняется и слева, и справа. Это лишь краткое сопоставление с нотацией Эйнштейна. Основная лабораторная продолжает писать $\sum$ явно и не вводит соглашения о ковариантных и контравариантных компонентах.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/

9.5. От XOR к произвольной MLP

XOR не является конечной целью курса. Это минимальный датасет, на котором видно, что несколько нелинейных скрытых характеристик могут представить зависимость, недоступную одной аффинной границе. В многослойном перцептроне, или MLP, этот же приём повторяется для большего числа признаков, скрытых нейронов и слоёв.

Пусть $\mathbf{h}_i^{(0)} = \mathbf{x}_i$ — исходный вектор признаков объекта, $m_0 = d$, а скрытый слой ℓ содержит m_ℓ нейронов. Тогда матрица его весов имеет форму $\mathbf{W}^{(\ell)} \in \mathbb{R}^{m_\ell \times (m_{\ell-1}+1)}$. Для слоя с номером $\ell \in \{1, \dots, K\}$ добавим постоянную координату к предыдущему состоянию и выполним аффинное преобразование с нелинейностью:

$$\tilde{\mathbf{h}}_i^{(\ell-1)} = \begin{pmatrix} 1 \\ \mathbf{h}_i^{(\ell-1)} \end{pmatrix}, \qquad \mathbf{r}_i^{(\ell)} = \mathbf{W}^{(\ell)} \tilde{\mathbf{h}}_i^{(\ell-1)},$$

$$\mathbf{h}_i^{(\ell)} = \varphi^{(\ell)} \left(\mathbf{r}_i^{(\ell)} \right).$$

Здесь $\mathbf{r}_i^{(\ell)}$ и $\mathbf{h}_i^{(\ell)}$ принадлежат $\mathbb{R}^{m_\ell}$, строка матрицы $\mathbf{W}^{(\ell)}$ задаёт веса одного нейрона слоя ℓ, а функция $\varphi^{(\ell)}$ применяется покоординатно. Для бинарного выхода после последнего скрытого слоя вводим $\tilde{\mathbf{v}} \in \mathbb{R}^{m_K+1}$ и сохраняем уже знакомую форму:

$$p_i = \sigma \left(\tilde{\mathbf{v}}^\top \tilde{\mathbf{h}}_i^{(K)} \right).$$

В прямом проходе MLP повторяет скалярные умножения, сложения и нелинейности для всех нейронов и слоёв. В обратном проходе не появляется новый принцип: reverse-mode проходит тот же DAG в обратном топологическом порядке, умножает входящий градиент на локальные производные операций и накапливает вклады в общих параметрах. Глубокая сеть создаёт более длинный граф, а не другое правило дифференцирования.

В практике этот маршрут применяется к вещественным векторам признаков и существенно большим датасетам. Учебная реализация намеренно не реализует произвольную многослойную сеть: в ней $K = 1$, скрытый слой содержит два tanh-нейрона, а бинарный выход использует sigmoid и BCE. Тензорная векторизация, многоклассовый выход и промышленный цикл обучения требуют дополнительных конструкций, но опираются на те же аффинные подграфы, нелинейности и накопление градиентов.

Итог. OR, AND и XOR образуют не набор разрозненных логических упражнений, а контролируемый маршрут к MLP: одна граница, обучение композиции, ограничение одной скрытой характеристики и объединение нескольких нелинейных признаков.

© Dmitry Vlasov 2026. All rights reserved.
Licensed under CC BY-NC-ND 4.0
https://creativecommons.org/licenses/by-nc-nd/4.0/